

CS571 Programming Languages

Spring 2026

6. Types



Agenda

- Overview
- Primitive types
- Composite types
- Pointers

Values and Types

- Values are type-less in hardware

0100 0000 0101 1000 0000 0000 0000 0000

- Floating point number: 3.375
 - Two 16-bit integer: 16472 and 0
 - 32-bit integer : 1,079,508,992
 - Four ASCII characters: @ X NUL NUL
- Operations expect certain values
 - `add` take integers, `fadd` take floats
- How to ensure operators get right **type** of values?

What Can Go Wrong?

- Operation on wrong values will produce garbage
 - float addition on two integers
 - assign a string to a float
 - pass an int to a function expecting a string
- All these errors are **type errors**

Type

- An abstraction of a set of values, and legal operations on these values
 - **int**: -2^{31} to $2^{31} - 1$, with operations $+$, $-$, $*$, $/$, ...
- Meaning of type
 - Denotational: a set of value
 - Constructive: set of primitive types, type constructors
 - Abstraction: an interface (set of operations)



Why Types

- Identify/prevent type errors
- Program organization/abstraction
- Documentation
- Support optimization

Type System

- A method or specification for associating types with variables, expressions, etc.
 - Type equivalence: are two types equivalent?
 - Type compatibility: can **int** be used as **float**?
 - Type safety: absence of type errors

Type Equivalence in C

- C uses name equivalence
- A type declaration corresponds to defining an implicit or explicit type name
- A **typedef** is simply an **alias** for an existing type and does not define a new type

```
struct {int i; double d;} x, y;  
struct {int i; double d;} a, b;  
typedef struct {  
    int i; double d;  
} T1, T2;  
T1 t1;  
T2 t2;  
x = y;           /* ok */  
a = b;           /* ok */  
t1 = t2;         /* ok */  
t1 = a;          /* type error */  
x = a;           /* type error */
```


Type Checking

- **Strongly typed**: all type errors caught by type checking
- **Weakly typed**: type checking may miss type errors
- **Static typed**: type checking happens at compile time
- **Dynamic typed**: type checking happens at run time
- **Type inference**: the type-checker infers types for variables

```
union U {int a; float p} u;  
float x = 1.0;  
u.a = 1;  
x = x + u.p;
```

C is weakly typed,
at compile time



Kinds of Types

- Primitive (Basic) Types
- Composite (Constructed) Types
 - Structs (Product Types)
 - Unions (Sum Types)
 - Arrays
 - Lists



Agenda

- Overview
- Primitive types
- Composite types
- Pointers



Primitive Types

- Numeric
 - bytes, integers, floats
- Booleans
- Characters
- Data types available on contemporary machines

Integers

- Length depends on language and compiler
- Representation: **two's complement** format

n	Binary representation of n
15	0000 0000 0000 0000 0000 0000 0000 1111
-n	Flip all bits, then add 1
5	0000 0000 0000 0000 0000 0000 0000 0101
-5	1111 1111 1111 1111 1111 1111 1111 1011
15-5?	0000 0000 0000 0000 0000 0000 0000 1010

Just add the binaries of 15 and -5, ignoring any overflow

Integers

Eight-bit integers

Bits ⇄	Unsigned value ⇄	Signed value (Two's complement) ⇄
0000 0000	0	0
0000 0001	1	1
0000 0010	2	2
0111 1110	126	126
0111 1111	127	127
1000 0000	128	-128
1000 0001	129	-127
1000 0010	130	-126
1111 1110	254	-2
1111 1111	255	-1

- The binary representations of n and $-n$ add up to 2^N
- fundamental arithmetic operations of $+$, $-$, and $*$ are identical to those for unsigned numbers

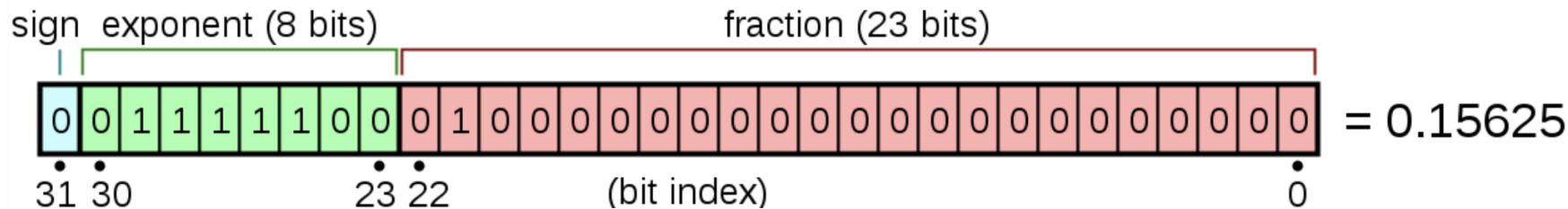
Floats

- Single precision (float): 32 bits
- Double precision (double): 64bits
- Due to the limited precision, floats are approximations of the real number they present

```
float z = 1.345+1.123;  
printf("%d\n", z==2.468);
```

Floats

IEEE745 Standard



$$\text{value} = (-1)^{\text{sign}} \times 2^{(E-127)} \times \left(1 + \sum_{i=1}^{23} b_{23-i} 2^{-i} \right)$$



Floats

- Can represent numbers in a big range, but with limited precision

> (+ 1.2 2.3 3.4)

6.9

$> (+ 3.4 \ 2.3 \ 1.2)$

6.89999999999999995

```
> (+ 1.0 1000000000000000000000000 -1000000000000000000000000)
```

0.0

> (+ 1000000000000000000000 -1000000000000000000000 1.0)

1.0

- $+$ operation not necessarily commutative

> (+ 1 100000000000000000000 -100000000000000000000)

1

Boolean

- Most languages: true or false
- In C: implemented as integer, 0 means false, all other values mean true
 - C99 introduced a new `_Bool` type, but it is effectively an integer that the compiler is permitted to store in a single bit
- In most implementations, a boolean value occupies more than one bit in memory (word is the basic unit of load/store)

Character

- All languages support ASCII code (7-bit)
 - Codes for lower-case letters are contiguous, as are the codes for upper-case letters and digits
 - In C :

```
int digitValue = digitChar - '0'
```
- Most modern language support Unicode
 - e.g., Java `char` uses UTF-16, a 16-bit char set

Character

- [Unicode](#) universal character set widely used by all modern computer systems to represent international characters
 - Over 1 million code points from `0x000000` to `0x10FFFF`
 - Characters often specified by name or by code-point as `U+xxxx`
 - Can be encoded using a fixed width (UTF-32) or variable width encoding (UTF-16 on windows, UTF-8 on unix and www)

UTF-8 Encoding

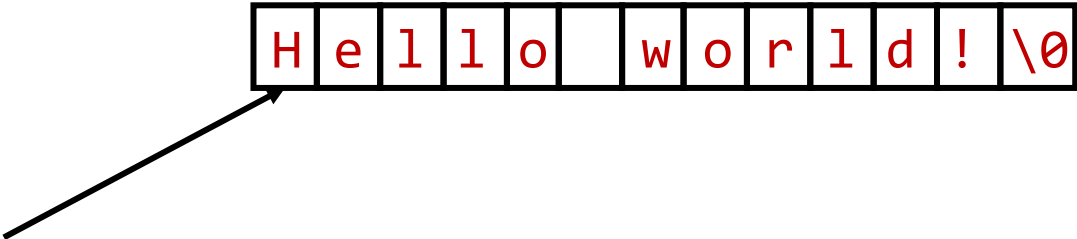
- Dominant encoding for www (97.9% of all web pages)
- Single byte encodings compatible with ASCII
- **Leading number of 1s** in first byte gives # of bytes in multi-byte sequences

Code point ↔ UTF-8 conversion

First code point	Last code point	Byte 1	Byte 2	Byte 3	Byte 4	Code points
U+0000	U+007F	0xxxxxxx				128
U+0080	U+07FF	110xxxxx	10xxxxxx			1920
U+0800	U+FFFF	1110xxxx	10xxxxxx	10xxxxxx		[a] 61440
U+10000	[b] U+10FFFF	11110xxx	10xxxxxx	10xxxxxx	10xxxxxx	1048576

Strings

- In C, strings are of type `char *`
- The pointer points to the start of the string.
- The string runs until the next null delimiter `'\0'` (equivalent to 0)



H	e	l	l	o		w	o	r	l	d	!	\0
---	---	---	---	---	--	---	---	---	---	---	---	----

`char *x = "Hello world!";`



C Strings Can Be Hazardous

```
char *str1 = "Hello World!";  
char[5] str2;  
strcpy(str2, str1);
```

Smashing The Stack For Fun And Profit: read [this paper](#)

Enumeration Types

- Provide names to a sequence of integral values
- Enumeration type improves readability
- In **C/C++**: implemented as integers

```
enum weekday {sun, mon, tue, wed, thu, fri, sat};  
// This is equivalent to:  
typedef int weekday;  
const weekday sun = 0, mon = 1, tue = 2,  
                wed = 3, thu = 4, fri = 5, sat = 6;
```



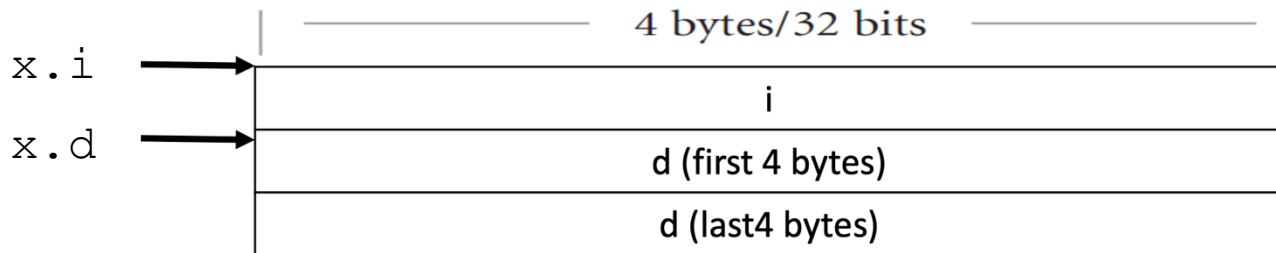

Agenda

- Overview
- Primitive types
- Composite types
 - Structs (Records, Product Types) and Unions (Sum Types)
 - Arrays and Lists
- Pointers

Records and Structures

```
struct id {  
    int i;  
    double d;  
};  
struct id x;  
x.i = 1, x.d = 3.14;
```

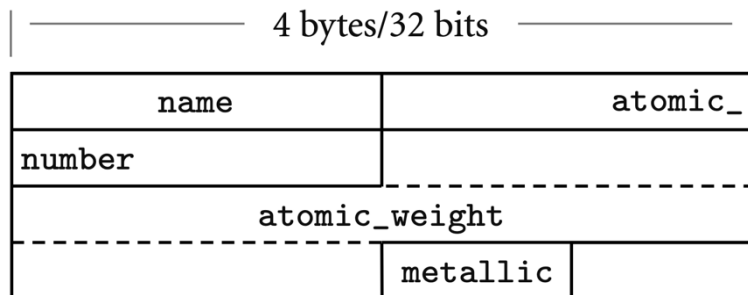
- Usually laid out **contiguously**
- Each field has a **separate** piece of memory



Memory Layout

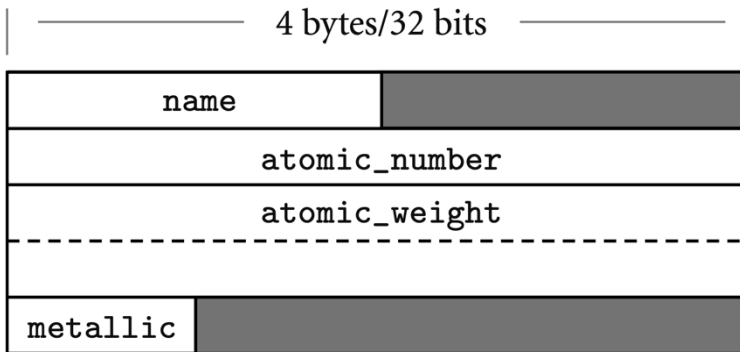
```
struct element {  
    char name[2];  
    int atomic_number;  
    double atomic_weight;  
    _Bool metallic;  
}
```

- No alignment leads to slow access



Memory Layout

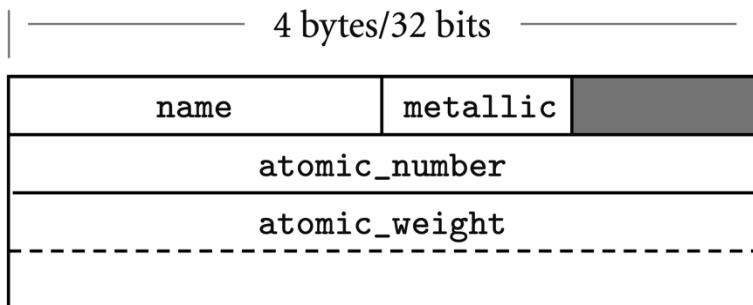
```
struct element {  
    char name[2];  
    int atomic_number;  
    double atomic_weight;  
    _Bool metallic;  
}
```



- Alignment makes fast access
- But leads to “holes”, therefore waste space
- Also complicate comparisons

Memory Layout

```
struct element {  
    char name[2];  
    int atomic_number;  
    double atomic_weight;  
    _Bool metallic;  
}
```



- Issues of record field order are intimately tied to implementation tradeoffs
- In most cases, reordering of fields is purely an implementation issue

Memory Layout of a Struct in C

- Compiler will respect declaration orders of the fields.
- Padding between fields to ensure every primitive type is stored at properly aligned memory address.
 - An 8-byte `double` will always be stored at a memory address that is a multiple of 8.
 - A 4-byte `int` will always be stored at a memory address that is a multiple of 4.
 - An array of 3 `int` will be stored at a memory address that is a multiple of 4 (not 12).
- There could be padding at the end of struct to ensure that the total size of the struct is a multiple of **its largest primitive type** size.



Memory Layout of a Struct in C

Example

```
struct s1 {  
    int i;  
    double d;  
    char c;  
};  
struct s1 x;
```

Memory Layout of a Struct in C

Example

```
struct s1 {  
    int i;  
    double d;  
    char c;  
};  
struct s1 x;
```

i	i	i	i
padding	padding	padding	padding
d	d	d	d
d	d	d	d
c	padding	padding	padding
padding	padding	padding	padding



Memory Layout of a Struct in C

Example

```
struct s1 {  
    int i;  
    double d;  
    char c;  
};  
struct s1 x;
```

Memory Layout of a Struct in C

Example

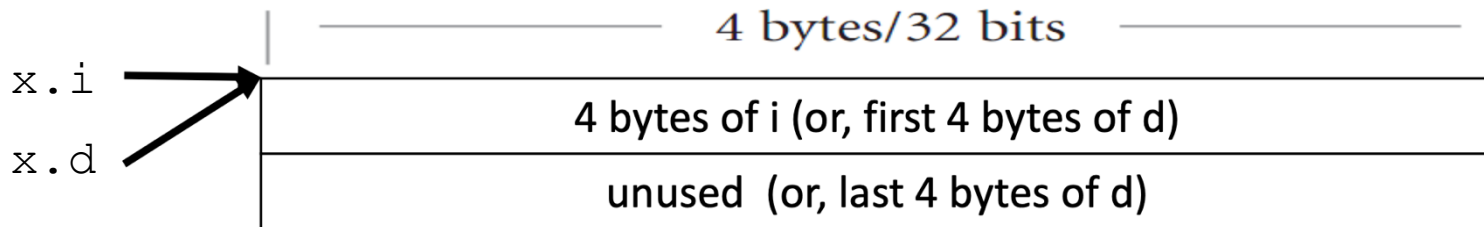
```
struct s2 {  
    int i[2];  
    char c;  
};
```

i[0]	i[0]	i[0]	i[0]
i[1]	i[1]	i[1]	i[1]
c	padding	padding	padding

Union Types

```
union id {  
    int i;  
    double d;};  
union id x;  
x.i = 1;  
y = 1.0 + x.d;
```

- Laid out in **shared** memory
- All fields share the same piece of memory



Memory Layout of Union in C

```
union u1 {  
    int i;  
    double d;  
    char c;  
};  
union s1 x;
```

Memory Layout of Union in C

```
union u1 {  
    int i;  
    double d;  
    char c;  
};  
union s1 x;
```

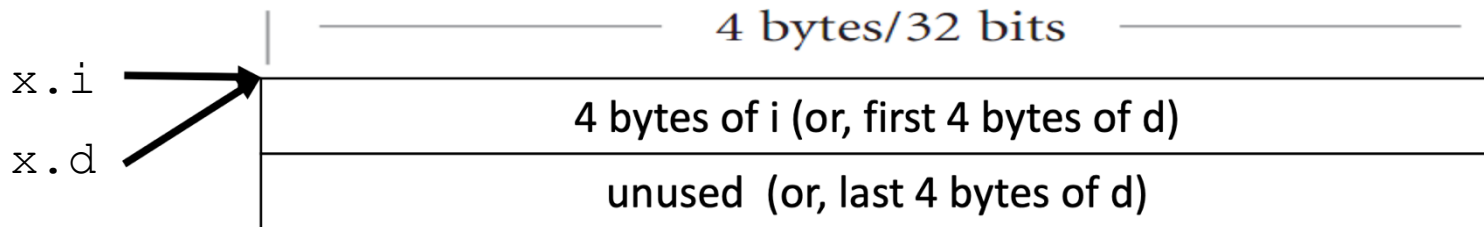
`sizeof(u1) = max(sizeof(int),
 sizeof(double),
 sizeof(char))`

i/d/c	i/d	i/d	i/d
d	d	d	d

Union Types

```
union id {  
    int i;  
    double d;};  
union id x;  
x.i = 1;  
y = 1.0 + x.d;
```

- **Not type safe:** `x.d` will read the binary `0x00000001, ??????????` as a double
- **How can we make it type safe?**



Discriminated Union Types

- A combination of a tag (like an enum) and a payload per possibility (like a union).

```
enum Kind {isInt, isFloat}
struct intorreal {
    enum Kind which;
    union U {int a; float p} u;
} ir;
float x = 1.0;
if (ir.which == isInt) ir.u.a = 1;
if (ir.which == isFloat) x = x + ir.u.p;
```

- Still not type safe: type system doesn't enforce tag check

Sum Types

- Many functional programming languages support type-safe **sum types**
- Type safe: type is checked under each case statement (the only way to read from a value with the sum type)

Haskell

tag

(possibly empty)
payload type

```
data intorreal = isInt Int | isFloat Float
-- given u has type intorreal
case u of
  isInt i  -> i + 1
  isFloat f -> f + 1.0
```


Sum Types are General

- A generalization of Enumeration type

Haskell

```
data day = Monday | Tuesday | Wednesday |  
         Thursday | Friday | Saturday | Sunday  
-- Given d has type day  
case d of  
  Monday -> ...  
  Tuesday -> ...  
  ...
```

Sum Types and Product Types

- Sum Types: alternation of types
- Product Types: concatenation of types

```
enum Color {Red, Blue}  
enum Shape {Circle, Rectangle}  
struct ColoredShape {enum color c; enum shape s}
```

Analogy:

Values of
Color

Values of
Shape

Values of
ColoredShape

$$\begin{aligned} & (\text{Red} + \text{Blue}) * (\text{Circle} + \text{Rectangle}) \\ = & (\text{Red} * \text{Circle}) + (\text{Red} * \text{Rectangle}) + (\text{Blue} * \text{Circle}) + (\text{Blue} * \text{Rectangle}) \end{aligned}$$

Algebraic Data Types (ADT)

- ADT: types are regarded as sets of values and set operations are used for constructing new types
- Primitive types: like Integer, String, etc.
- **Product types**: If $T_1, \dots, T_n$ are types, so is their cartesian product $T_1 \times \dots \times T_n$
 - Results in **tuples** like `<String, Integer, Integer>`
- **Sum types**: If $T_1, \dots, T_n$ are types, so is their union $T_1 \cup \dots \cup T_n$
 - Referred to as tagged or disjoint unions.

Dynamic Typed Languages

- The Truth about Dynamic Typed Languages: **One big sum type**
 - Everything is of the same sum type, something like “Object”
 - Each value carries with it a tag
- Object’s type is checked during run time through checking this type tag.
 - Type is used to find out how to interpret the object’s data
 - Type is used to check if an operation is legal



Agenda

- Overview
- Primitive types
- Composite types
 - Structs (Records, Product Types) and Unions (Sum Types)
 - Arrays and Lists
- Pointers

Array

- Lifetime and array size
 - Global lifetime, Static shape
 - Local lifetime, Static shape
 - Local lifetime, Dynamic shape

```
int A[10];
```

```
int f() {  
    int A[10];  
    ...}
```

```
int f(int n) {  
    int A[n];  
    ...}
```

Bound Checking

- Is it done?
- How is it done?
- When is it done?

```
int f() {  
    int A[10];  
    ...  
    A[e] = 1;  
    ...}
```

```
int f(int n) {  
    int A[n];  
    ...  
    A[e] = 1;  
    ...}
```

C Array

- Static/Stack/Heap allocated
- Size statically/dynamically determined
- Array bounds not checked (buffer overflow)



Java Array

- Heap allocated
- Size dynamically determined
- Array size is part of stored data (**Dope Vector**)
- Array bounds checked

Dope Vectors Example

4	int	10	A[0]	A[1]	...	A[9]
---	-----	----	------	------	-----	------

elem. size elem. type array size

↑
pointer
of A

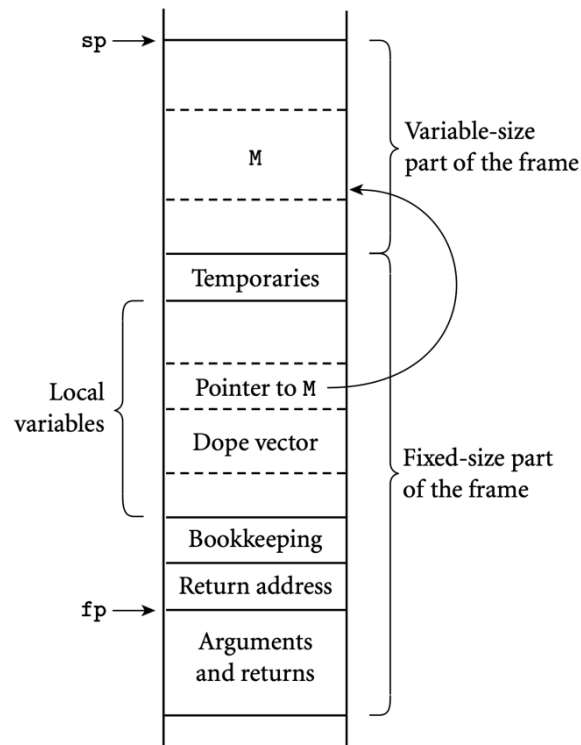
- Address of A[i]? $A + 4*i$
- Bound check? $0 \leq i < 10$
- Benefit: the array may change dynamically

Stack vs Heap Allocation

- Size statically determined; fixed
 - Allocation in the usual way
- Size dynamically determined; fixed
 - May still be allocated in the stack
 - Need shape information at run time (dope vector)
- Size may change
 - Heap allocated

Elaboration-time Allocation of Arrays

```
//C99:  
void foo(int size) {  
    int M[size][size];  
    ...  
}
```



Memory Layout

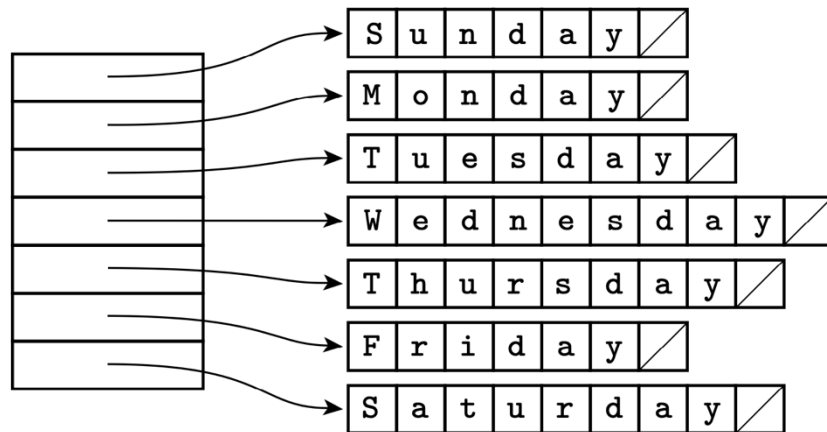
Contiguous array allocation

```
char days[][10] = {
    "Sunday", "Monday", "Tuesday",
    "Wednesday", "Thursday",
    "Friday", "Saturday"
};
```

S	u	n	d	a	y				
M	o	n	d	a	y				
T	u	e	s	d	a	y			
W	e	d	n	e	s	d	a	y	
T	h	u	r	s	d	a	y		
F	r	i	d	a	y				
S	a	t	u	r	d	a	y		

Row pointer layout

```
char *days[] = {
    "Sunday", "Monday", "Tuesday",
    "Wednesday", "Thursday",
    "Friday", "Saturday"
};
```

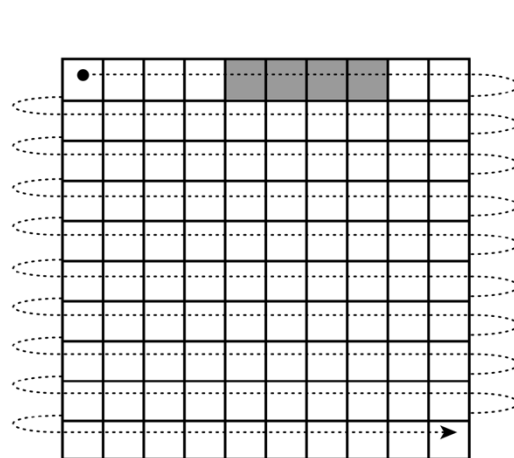


Memory Layout

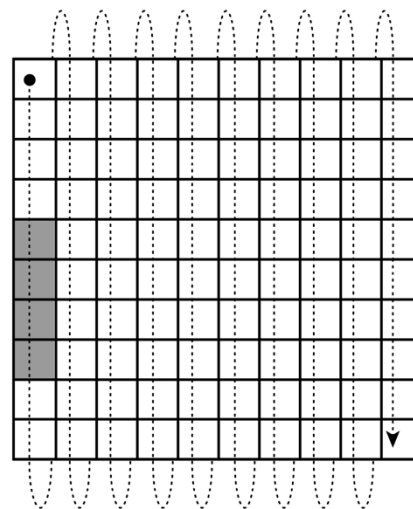
- 1-D arrays
- 2-D arrays



```
int A[][] = new int[10][10]
```



Row-major order



Column-major order

Address Calculation (Row Major)

4	int	10	100	A[0][0]	A[0][1]	...	A[9][99]
---	-----	----	-----	---------	---------	-----	----------

elem. size elem. type row col

↑
pointer
of A

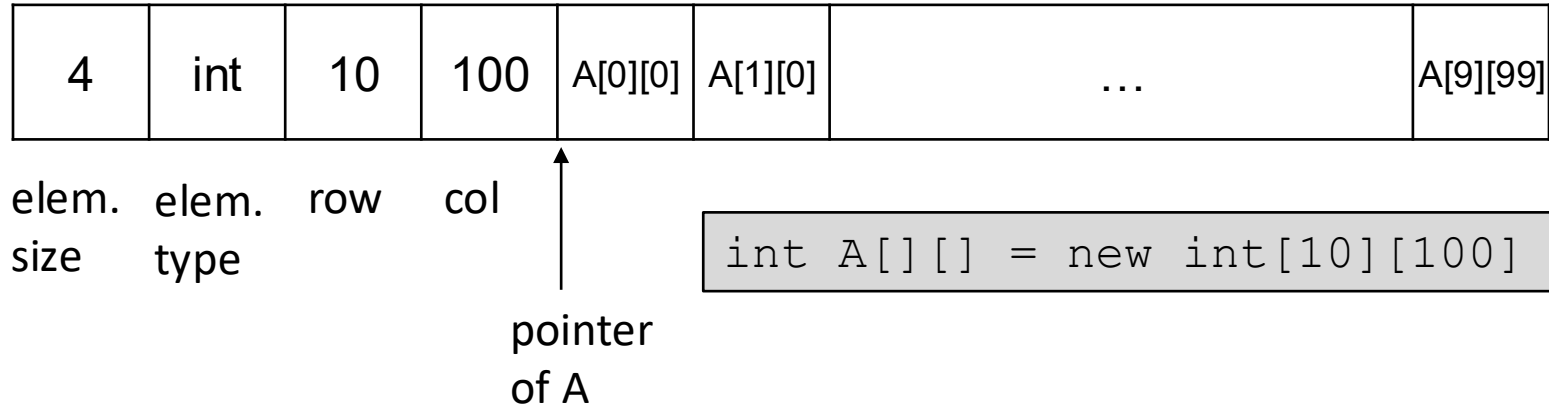
```
int A[][] = new int[10][100]
```

- Address of $A[i][j]$?
- Bound check?

$$A + 4 * (i * 100 + j)$$

$$0 \leq i < 10, 0 \leq j < 100$$

Address Calculation (Column Major)



- Address of $A[i][j]$?
- Bound check?

$$A + 4 * (j * 10 + i)$$

$$0 \leq i < 10, 0 \leq j < 100$$

Lists vs. Arrays

- Lists: contain a sequence of elements (like arrays)
- No notion of mapping or **indexing**
- Defined recursively as either an empty list (**base case**) or a pair consisting of a **head** element and a reference to a sublist (**tail**)
- Always of variable length
- **Fundamental in most functional languages**



Agenda

- Overview
- Primitive types
- Composite types
- Pointers

Pointers

- What are they?
 - A set of memory addresses and operations on them
 - **Values**: legal addresses, and a special value: **nil**
 - **Operations**: assignment, dereferencing, arithmetic
- Uses:
 - Indirect addressing (access arbitrary address)
 - Manage dynamic storage (heap)

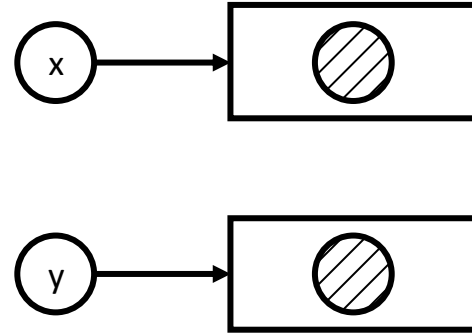
```
int x = 20;  
int* p = &x;  
int y = *p
```

```
int a[3] = {1, 2, 3};  
int x = *(a+1) //same as a[1]
```

C Pointer Assignment

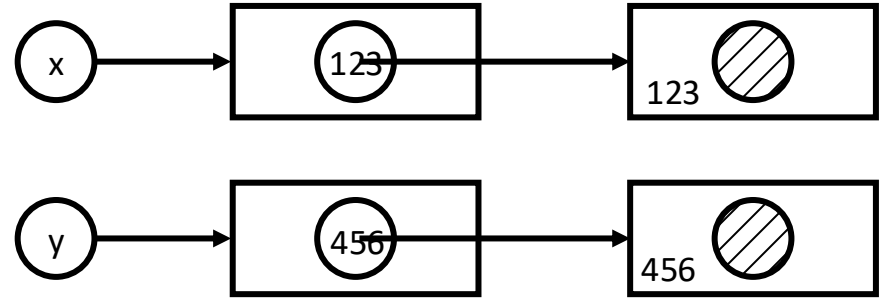
➔

```
int *x, *y;  
x = (int *)malloc(sizeof(int));  
y = (int *)malloc(sizeof(int));  
*x = 1;  
*y = 0;  
y = x;
```



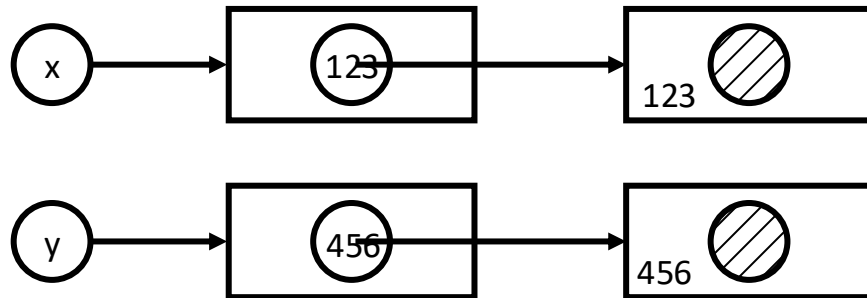
C Pointer Assignment

```
int *x, *y;  
x = (int *)malloc(sizeof(int));  
y = (int *)malloc(sizeof(int));  
→ *x = 1;  
  *y = 0;  
  y = x;
```



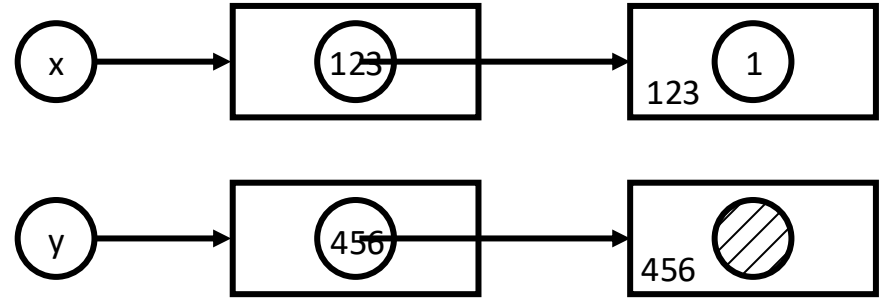
C Pointer Assignment

```
int *x, *y;  
x = (int *)malloc(sizeof(int));  
y = (int *)malloc(sizeof(int));  
→ *x = 1;  
  *y = 0;  
  y = x;
```



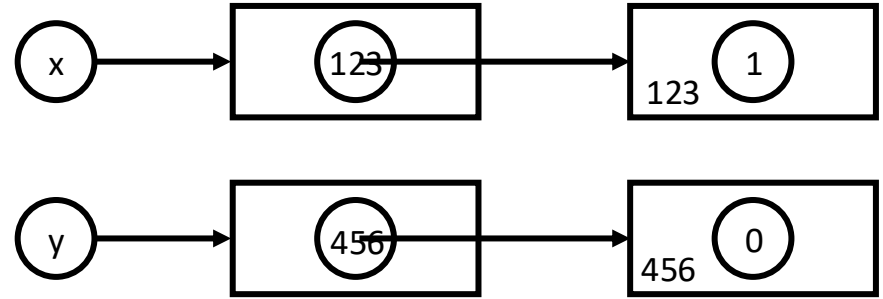
C Pointer Assignment

```
int *x, *y;  
x = (int *)malloc(sizeof(int));  
y = (int *)malloc(sizeof(int));  
→ *x = 1;  
  *y = 0;  
  y = x;
```



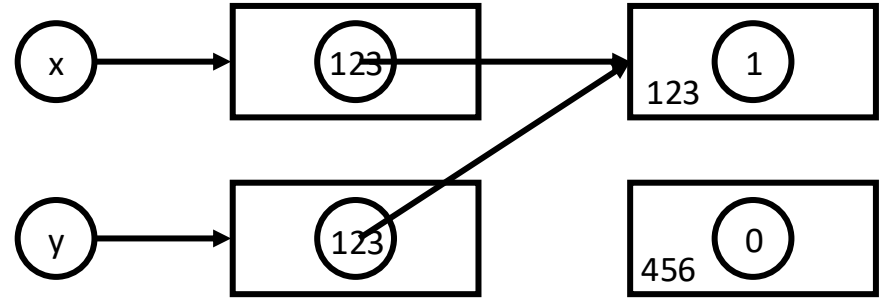
C Pointer Assignment

```
int *x, *y;  
x = (int *)malloc(sizeof(int));  
y = (int *)malloc(sizeof(int));  
*x = 1;  
*y = 0;  
→ y = x;
```



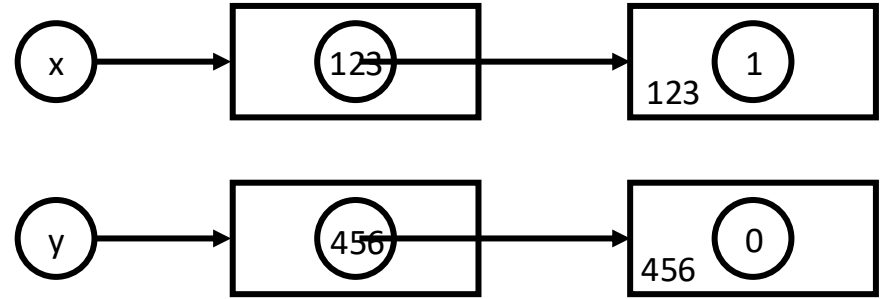
C Pointer Assignment

```
int *x, *y;  
x = (int *)malloc(sizeof(int));  
y = (int *)malloc(sizeof(int));  
*x = 1;  
*y = 0;  
→ y = x;
```



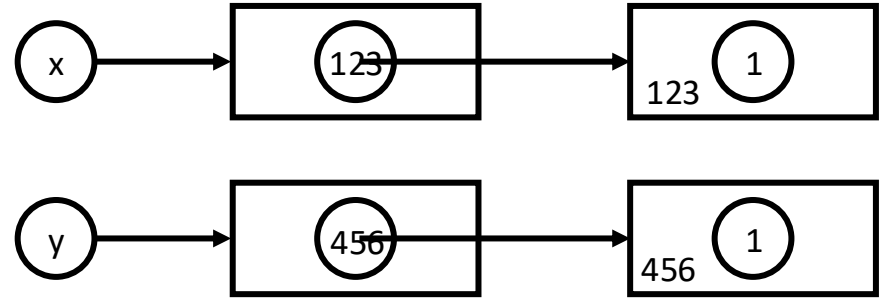
C Pointer Assignment Take 2

```
int *x, *y;  
x = (int *)malloc(sizeof(int));  
y = (int *)malloc(sizeof(int));  
*x = 1;  
*y = 0;  
→ *y = *x;
```



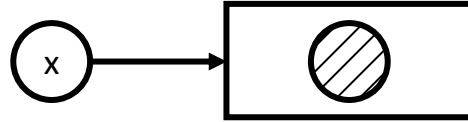
C Pointer Assignment Take 2

```
int *x, *y;  
x = (int *)malloc(sizeof(int));  
y = (int *)malloc(sizeof(int));  
*x = 1;  
*y = 0;  
→ *y = *x;
```



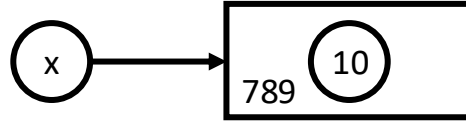
C Pointer Assignment: Take 3

→ `int x;`
`x = 10;`
`int *y = &x;`



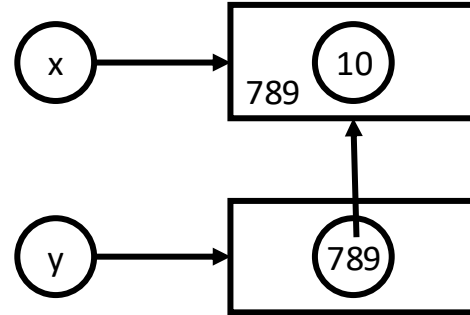
C Pointer Assignment: Take 3

```
int x;  
x = 10;  
→ int *y = &x;
```



C Pointer Assignment: Take 3

```
int x;  
x = 10;  
→ int *y = &x;
```



Pointer Arithmetic

- Pointer is a type!
 - Pointers are not just memory addresses
 - May store base address + offset, or dope vector
- Pointer arithmetic:
 - If `p` is of type `int*`, then `p + 3` is also of type `int*`
 - It points to the memory `3*sizeof(int)` bytes after `p`

Pointers and Arrays

- Type: `int[]` is equivalent to `int*`
- `[]`: subscript operator, defined in terms of pointer arithmetic
 - `A[B]` is defined to be `*(A+B)`, thus `A[3]` is equivalent to `3[A]`
- A pointer declaration must allow the compiler to determine the size of the objects referred to by the pointer
 - 2D-array with row-pointer layout: `int *a[]` or `int **a`
 - 2D-array with contiguous layout: `int a[][m]` or `int (*a)[m]`
 - Invalid: `int a[][]` or `int (*a)[]`
 - # of elements in the array a: `sizeof(a) / sizeof(a[0])`

Function pointers

- A function pointer can be stored in a value with type
<return value> (*<variable name>) (<parameter1>, <parameter2>, ...)

- This enables dynamically
choosing which function to call

```
int callback1(float x){ ... }  
int callback2(float x){ ... }  
...  
int (*cb)(float);  
if (x >= 0) cb = callback1;  
else cb = callback2;  
...  
cb(0.3);
```

References

- Restricted pointers: cannot be used as value or operated in any way
 - Not directly visible to the programmer
 - No explicit data type
- Uses: Alias of existing variables

```
double r = 2.3;  
double &s = r; //s is an alias of r (share memory)  
double *p = &r; //p has value: address of r  
s += 1; *p += 1;
```

Pointers vs. References

- Pointers: `int *p;`
- References: `int &p;`
- Value Model vs. Reference Model: **A = B**
 - Value model: the value of **B** is copied to **A**
 - Reference model: **A** is an alias of **B** (same memory)
 - Java: primitive types follow value model; objects follow reference model

The Nil Pointer Problem

- The concept of a “null reference” was invented in 1965 by Sir Tony Hoare (Sir Charles Antony Richard Hoare, C.A.R Hoare)
 - In Hoare’s own words, he introduced the null reference “simply because it was easy to implement.”
- In a famous 2009 talk, he called it his “billion-dollar mistake”

“I call it my billion-dollar mistake. It was the invention of the null reference in 1965. ... This has led to innumerable errors, vulnerabilities, and system crashes, which have probably caused a billion dollars of pain and damage in the last forty years.”

The Nil Pointer Problem

- One mitigation: use sum types

Haskell

```
data Maybe Int = Nothing | Just Int
-- given p has type Maybe Int
case p of
  Nothing -> .. nil value ..
  Just i -> .. valid value ..
```

SML

```
datatype int option = NONE | SOME of int
-- given p has type int option
case p of
  None => .. Nil value ..
| SOME i => .. Valid value ..
```